

The Complete Beginner-to-Expert Guide: SA-UNetv2 Retinal Vessel Segmentation

For: Arth Patel — B.Tech Project Defense

Assumes: Zero ML/DL background. Everything is explained from the ground up.

Goal: After reading this, you should be able to answer ANY question your professor throws at you.

PART A: THE FOUNDATIONS (What is Deep Learning, CNNs, Segmentation?)

A1. What Even Is a Neural Network? (The Absolute Basics)

Imagine you have a photograph of a retina (the back of the eye). You want a computer to look at it and color every blood vessel pixel in green, and leave everything else black. That's the task.

A **neural network** is basically a giant math function with millions of tiny adjustable knobs (called **weights** or **parameters**). You feed it an image, it does a bunch of math, and it spits out an answer.

The key insight: When you first create the network, all the knobs are set randomly, so it gives garbage output. But then you show it thousands of examples where you already know the correct answer (these are called **ground truth labels** — in our case, a human expert doctor has already traced every blood vessel by hand). Each time the network gets it wrong, you nudge the knobs slightly in the direction that would have made the answer more correct. Do this millions of times, and the network "learns" to recognize blood vessels.

This process of nudging knobs is called **training**. The nudging is done by a mathematical technique called **gradient descent** (specifically **backpropagation**).

Analogy: The Blindfolded Sculptor

Imagine a blindfolded person sculpting a statue. They can't see the result, but a friend tells them "you carved too much on the left, not enough on the right." Each time, they

adjust slightly. After thousands of adjustments, the statue looks great. The "friend" is the **loss function** (more on this later), and the "adjustments" are **gradient updates**.

A2. What Is a Convolutional Neural Network (CNN)?

Regular neural networks treat every pixel independently. But images have **spatial structure** — a pixel's meaning depends heavily on its neighbors. A red pixel alone means nothing, but a cluster of dark pixels in a line means "blood vessel."

A **Convolutional Neural Network (CNN)** uses small sliding windows (called **filters** or **kernels**) that scan across the image. Each filter is a tiny grid (e.g., 3×3 pixels) that looks at a small patch of the image at a time.

How a Convolution Works (Step by Step):

1. Take a 3×3 filter (9 numbers, initially random).
2. Place it on the top-left corner of the image.
3. Multiply each filter number with the corresponding pixel underneath.
4. Sum all 9 products → this gives ONE output number.
5. Slide the filter one pixel to the right and repeat.
6. Keep sliding across the entire image → you get an **output feature map**.

What Does This Actually Learn?

- One filter might learn to detect **horizontal edges** (transitions from dark to light going left-to-right).
- Another filter might learn **vertical edges**.
- A third might detect **curved lines** (like blood vessels!).
- Deeper filters combine these basic detections into complex patterns: "this curved edge next to this junction = a vessel bifurcation."

Analogy: The Detective with a Magnifying Glass

Imagine a detective scanning a crime scene with a magnifying glass. They can only see a 3×3 inch patch at a time, but by scanning the entire scene, they build a mental map of where all the clues are. Each filter is like a different type of magnifying glass — one highlights fingerprints, another highlights blood stains, another highlights footprints.

A3. What Are Channels / Feature Maps?

When you look at a color photo, it has 3 **channels**: Red, Green, Blue (RGB). Each channel is a 2D grid of numbers (pixel intensities from 0 to 255).

After the first convolution layer with, say, 16 filters, you get **16 output channels** (also called **feature maps**). Each feature map highlights a different pattern the filter detected.

So when we say "16 channels → 32 channels → 48 channels → 64 channels", we mean:

- Layer 1 produces 16 different pattern maps.
- Layer 2 combines those 16 maps and produces 32 even more complex pattern maps.
- Layer 3 produces 48 even higher-level pattern maps.
- Layer 4 (bottleneck) produces 64 of the most abstract, semantic pattern maps.

Analogy: Instagram Filters

Think of 16 channels as applying 16 different Instagram filters to the same photo. One shows only edges, one shows only bright spots, one shows only dark regions, etc. The next layer then looks at ALL 16 filtered versions simultaneously to find even more complex patterns.

A4. What Is Pooling? (MaxPool)

After convolutions detect features, we want to:

1. **Reduce the image size** (so computation stays manageable).
2. **Make detections position-invariant** (a vessel in the top-left should be recognized the same as one in the bottom-right).

Max Pooling with a 2×2 window:

- Take every 2×2 block of pixels.
- Keep only the maximum value.
- Discard the other 3.
- Result: image shrinks to half its width and half its height.

After 3 max-pooling operations:

- $592 \times 592 \rightarrow 296 \times 296 \rightarrow 148 \times 148 \rightarrow 74 \times 74$

This is why we need dimensions divisible by 8 (2^3). If the image is 584×565 , dividing by 2 three times gives non-integer sizes. So we **pad** it to 592×592 first.

Analogy: Zooming Out on Google Maps

When you zoom out on Google Maps, you lose the street names and tiny details, but you gain the ability to see the overall road network and city structure. MaxPool does the same — sacrifices fine details to see the big picture.

A5. What Is Image Segmentation?

There are three main tasks in computer vision:

1. **Classification:** "Is there a cat in this image?" → Yes/No.
2. **Object Detection:** "Where is the cat?" → Draws a bounding box.
3. **Segmentation:** "Which exact pixels belong to the cat?" → Colors every cat pixel.

Our task is **binary semantic segmentation**: for every single pixel, decide: **is this pixel a blood vessel (1) or not (0)?**

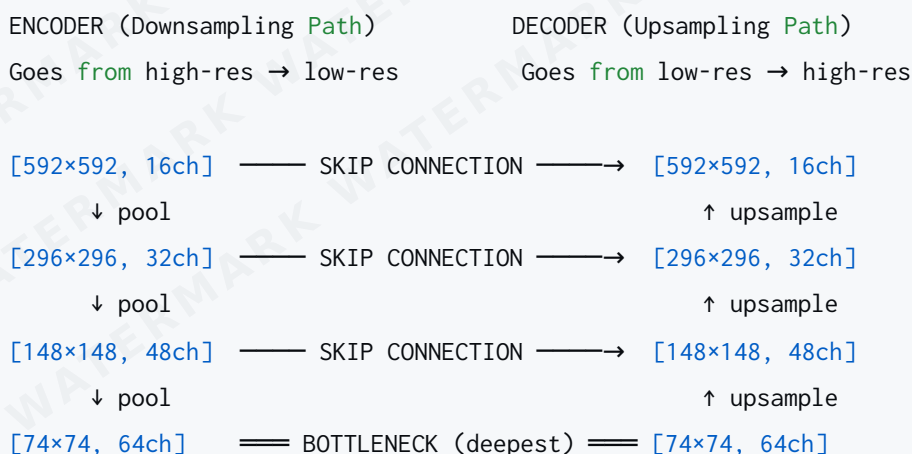
The output is the same size as the input image, but instead of 3 color channels, it has 1 channel with values between 0.0 and 1.0 (the model's confidence that each pixel is a vessel).

A6. What Is U-Net and Why Was It Invented?

U-Net (2015, MICCAI) was specifically designed for medical image segmentation. The key problem it solves:

The Dilemma: To understand **WHAT** is in the image (semantic meaning), you need to go deep (many pooling layers → small feature maps → abstract understanding). But to know **WHERE** things are at pixel precision, you need high-resolution spatial detail — which pooling destroys.

U-Net's Solution: The Encoder-Decoder with Skip Connections



- **Encoder (left side):** Gradually shrinks the image while extracting increasingly abstract features. At the bottom, it "understands" the image semantically (knows where vessels are roughly), but has lost pixel-precision.
- **Decoder (right side):** Gradually upsamples (enlarges) back to the original resolution. But upsampling alone produces blurry outputs.
- **Skip Connections (horizontal arrows):** Copy the high-resolution encoder features and paste them into the decoder. This gives the decoder both **semantic understanding** (from the upsampled deep features) AND **spatial precision** (from the copied encoder features).

Analogy: Writing an Essay

The **encoder** is like reading a book and making increasingly condensed notes: first page-by-page notes, then chapter summaries, then a one-paragraph thesis. The **decoder** is like writing the essay from that thesis, expanding back out. But the thesis alone is too vague, so you keep your original page notes nearby (**skip connections**) to fill in specific quotes and details.

Why is it called "U-Net"?

Because the architecture diagram looks like the letter **U** — it goes down (encoder), hits the bottom (bottleneck), and comes back up (decoder).

PART B: SA-UNetv2 — WHAT MAKES IT SPECIAL?

B1. The Problem with Existing Models

Model	Year	Parameters	Problem
U-Net	2015	31 million	Way too many parameters. Slow. Overfits on small medical datasets.
AG-Net	2019	9.3 million	Still very large.
MamUNet	2025	16.8 million	Extremely heavy. Needs powerful GPUs.

Model	Year	Parameters	Problem
RetinalLiteNet	2024	0.066 million	Ultra-small, but accuracy drops significantly.
SA-UNet v1	2020	0.54 million	Good balance, but still has redundant channels.
SA-UNetv2	2026	0.26 million	52% smaller than v1, yet more accurate!

SA-UNetv2 achieves the seemingly impossible: **fewer parameters AND higher accuracy**. How?

B2. The Three Key Innovations (The "Secret Sauce")

Innovation 1: Modernized Conv Block (DropBlock + GroupNorm + SiLU)

Innovation 2: Cross-Scale Spatial Attention (CSA) on Skip Connections

Innovation 3: Compound BCE + MCC Loss Function

Let's go deep into each one.

B3. Innovation 1: The Conv Block — Every Single Sub-Layer Explained

The old standard was: Conv → BatchNorm → ReLU

SA-UNetv2 uses: Conv → DropBlock → GroupNorm → SiLU

Every part was replaced for a specific reason. Let's understand each.

B3.1: The Convolution Layer (Conv 3×3)

This is the standard 3×3 convolution described in Section A2. Nothing new here — it's the same sliding-window feature extraction.

One small detail: there is **no bias term** in the convolution. Why? Because the GroupNorm layer that follows already has its own bias parameter, so adding one in the convolution would be redundant (two biases doing the same job).

B3.2: DropBlock — Why Standard Dropout Fails for Images

First, what is Dropout?

During training, **Dropout** randomly "turns off" some neurons (sets their output to zero). This prevents the network from relying too heavily on any single neuron, forcing it to spread knowledge across many neurons. It's a regularization technique (prevents overfitting).

Standard Dropout randomly zeros out individual, isolated pixels in the feature map. For example:

Original feature map:	After standard Dropout (random pixels killed):
[5 3 7 2 8]	[5 0 7 2 8]
[1 9 4 6 3]	[1 9 0 6 3]
[7 2 8 1 5]	[7 2 8 0 5]
[4 6 3 9 2]	[0 6 3 9 2]

See the problem? The killed pixels (the zeros) are scattered randomly. But convolution uses a 3×3 window — it looks at 9 neighbors at once. Even if one pixel is zeroed, the surrounding 8 neighbors still carry the information. The convolution just "fills in the gap" from context. So standard Dropout **has almost no regularization effect** in convolutional layers!

DropBlock: The Fix

DropBlock zeros out an entire contiguous 7×7 square region:

Original feature map:	After DropBlock (entire 7×7 region killed):
[5 3 7 2 8 1 4 9]	[5 3 0 0 0 0 0 0]
[1 9 4 6 3 7 2 5]	[1 9 0 0 0 0 0 0]
[7 2 8 1 5 3 6 4]	[7 2 0 0 0 0 0 0]
[4 6 3 9 2 8 1 7]	[4 6 0 0 0 0 0 0]
[8 1 5 4 7 2 9 3]	[8 1 0 0 0 0 0 0]
[3 7 2 6 1 5 4 8]	[3 7 0 0 0 0 0 0]
[9 4 6 3 8 1 7 2]	[9 4 0 0 0 0 0 0]

Now there's a **huge hole** that the convolution CANNOT fill in from neighbors (because ALL the neighbors are also zero). The network is forced to learn the same pattern from OTHER parts of the image.

Parameters in our model:

- `block_size = 7`: Each dropped region is a 7×7 square.
- `rate = 0.15`: 15% of all spatial positions are selected as centers for dropped blocks.

Analogy: Studying for an Exam

Standard Dropout is like covering individual random words in your textbook with tape. You can still read the sentence because you can guess the missing word from context.

DropBlock is like ripping out entire paragraphs. Now you're forced to learn the concept from OTHER chapters, making your understanding more robust and generalized.

Important: DropBlock is ONLY active during training. During testing/inference, nothing is dropped — the full feature map is used.

B3.3: Group Normalization — Why BatchNorm Breaks with Small Batches

First, what is Normalization and why do we need it?

As data flows through many layers, the numbers can become very large or very small (a phenomenon called **internal covariate shift**). This makes training unstable — the gradient updates become erratic.

Normalization standardizes the numbers at each layer to have **mean** ≈ 0 and **standard deviation** ≈ 1 , keeping everything in a well-behaved numerical range.

Batch Normalization (the standard, but problematic)

BatchNorm calculates the mean and variance **across all images in the current batch**.

Example: If batch size = 32 (32 images processed together), BatchNorm computes the average pixel value across all 32 images. With 32 samples, the statistics (mean, variance) are reliable.

The problem in medical imaging: Our retinal images are $592 \times 592 \times 3$ — each one is huge. GPU memory can only fit a few at a time:

- DRIVE: batch size = 8
- STARE: batch size = 2

With only 2 images in a batch, the computed mean and variance are **extremely noisy and unreliable** (imagine estimating the average height of all humans by measuring only 2 people). This noise causes BatchNorm to destabilize training.

Group Normalization (the solution)

GroupNorm doesn't look at other images at all. Instead, it operates on a **single image** independently:

1. Take the feature map of one image (e.g., 32 channels, each a 2D spatial map).
2. Divide the 32 channels into $G = 8$ groups (4 channels per group).
3. Within each group, compute mean and variance across all spatial positions.
4. Normalize using those per-group statistics.

Since it only looks at one image at a time, it doesn't matter if the batch has 2 images or 200 images — the normalization quality is identical.

Analogy: Grading Exams

BatchNorm is like grading on a curve relative to the whole class. If your class only has 2 students, the curve is meaningless.

GroupNorm is like grading each student based on their own performance across different sections of the exam. It works perfectly regardless of class size.

Why groups = 8? The channel counts are [16, 32, 48, 64]. All of these are evenly divisible by 8 (giving groups of size 2, 4, 6, 8), so there's no remainder or wasted channels.

B3.4: SiLU Activation — Why ReLU Has Problems

First, what is an activation function?

After convolution and normalization, we apply a non-linear function to the output. Why? Because without non-linearity, stacking multiple linear layers is equivalent to a single linear layer (matrix multiplication is commutative). Non-linearity is what gives neural networks the power to learn complex, curved, non-obvious patterns.

ReLU (Rectified Linear Unit) — The Old Standard

$$\text{ReLU}(x) = \max(0, x)$$

- If input is positive → output equals input (pass through unchanged).
- If input is negative → output is exactly **zero** (killed).

Problem: The "Dying ReLU" Problem Once a neuron's output becomes negative, ReLU kills it to zero. The gradient (the signal used to update weights) for zero output is also **exactly zero**. So the weight never gets updated again — the neuron is permanently "dead."

For blood vessel detection, many subtle, low-contrast capillary features produce small, slightly negative activations. ReLU kills ALL of these, losing critical information about faint vessels.

SiLU (Sigmoid Linear Unit) — The Replacement

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$

Where $\sigma(x) = \frac{1}{1+e^{-x}}$ is the **sigmoid function** (an S-shaped curve that smoothly maps any number to a value between 0 and 1).

Key properties of SiLU:

- For large positive x : $\text{SiLU} \approx x$ (similar to ReLU).
- For large negative x : $\text{SiLU} \approx 0$ (similar to ReLU).
- For small negative x (like $x = -1$): SiLU gives a **small non-zero negative value** (≈ -0.27), NOT zero!
- The curve is **smooth everywhere** – no sharp corner like ReLU at $x = 0$.

This means:

1. **Neurons never fully die.** Small negative activations still produce non-zero output and non-zero gradients.
2. **Gradients flow smoothly** through the network, allowing it to learn subtle, low-contrast features like faint capillary boundaries.

Analogy: A Volume Knob

ReLU is like a light switch – either fully ON or fully OFF. If a vessel signal is even slightly "negative" (maybe just noisy), the switch turns off and the vessel is lost forever.

SiLU is like a dimmer/volume knob – it smoothly reduces the signal for slightly negative values instead of killing it entirely. This preserves faint vessel signals that would otherwise be lost.

B4. Innovation 2: Attention Mechanisms – SA and CSA

B4.1: What Is "Attention" in Deep Learning?

In everyday life, when you look at a photo, you don't examine every pixel with equal focus. Your eyes automatically focus on important regions (a person's face, a road sign) and ignore unimportant regions (plain sky, flat road).

Attention mechanisms teach the neural network to do the same thing: generate a "importance map" (values 0 to 1) that tells the network WHERE to focus.

- Importance = 1.0 → "Pay maximum attention here" (likely a vessel).
- Importance = 0.0 → "Ignore this completely" (definitely background).

- Importance = 0.5 → "Moderate attention" (uncertain region).

This importance map is multiplied element-wise with the feature map. High-attention regions pass through unchanged; low-attention regions get suppressed (multiplied toward zero).

Analogy: A Highlighter Pen

Attention is like going through a textbook with a highlighter. You highlight the important sentences (vessels) and leave the rest unmarked (background). When you study later, you focus only on the highlighted parts.

B4.2: Spatial Attention (SA) – The Bottleneck Module

Location: Inside the bottleneck (the deepest, smallest layer of the U-Net).

Why here? The bottleneck has the most abstract, semantically rich features, but also the lowest spatial resolution (74×74). At this stage, the network has a "rough idea" of where vessels are, but needs to sharpen that understanding.

Step-by-step operation:

1. **Input:** A feature map F with shape $74 \times 74 \times 64$ (64 channels).
2. **Channel-wise Average Pooling:** At each spatial position (h, w) , average all 64 channel values into one number:

$$A(h, w) = \frac{1}{64} \sum_{c=1}^{64} F(h, w, c)$$

Result: A single-channel map $A \in \mathbb{R}^{74 \times 74 \times 1}$.

Intuition: This squashes "what patterns were detected" into "how much total activity is at this location." High-activity locations probably contain vessels.

3. **7×7 Convolution:** Apply a 7×7 convolution filter to this activity map. This looks at a neighborhood of 7×7 locations to determine if the activity pattern looks like a vessel vs. noise.

Why 7×7 and not 3×3 ? A larger window captures more spatial context. A 3×3 window can only see immediate neighbors, but vessels are elongated structures — you need to look further to recognize vessel-like patterns.

4. **Sigmoid Activation:** Pass the convolution output through a sigmoid function to squash values to $[0, 1]$. This creates the **attention mask**.

5. **Element-wise Multiplication:**

$$F_{\text{out}} = F \odot \text{AttentionMask}$$

Every channel of the original feature map F is multiplied by this mask. Vessel regions remain strong; background regions get suppressed.

Parameters: Just $7 \times 7 \times 1 \times 1 = 49$ parameters (essentially negligible).

B4.3: Cross-Scale Spatial Attention (CSA) – The Skip Connection Module

This is the paper's **most novel contribution** and the one your professor will most likely ask about.

Location: On every skip connection (3 of them – between encoder levels 1, 2, 3 and their corresponding decoder levels).

The Problem CSA Solves: The "Semantic Gap"

In standard U-Net, skip connections simply copy-paste encoder features directly into the decoder:

Encoder features (Level 2) — copy-paste —> Decoder (Level 2)

But think about what each side "knows":

- **Encoder features:** Very precise spatial detail (exact pixel edges), but NO understanding of what's a vessel vs. what's noise or a lesion.
- **Decoder features (upsampled):** Strong understanding of WHAT is a vessel, but blurry, imprecise spatial location.

When you naively concatenate these, you get a mess – the decoder has to figure out which encoder pixels are relevant and which are noise. This is called the **semantic gap**.

How CSA Bridges the Gap

CSA uses the decoder's semantic knowledge to **guide which encoder features should pass through**:

Step-by-step operation:

1. Two Inputs:

- F_e = Encoder feature map (e.g., $296 \times 296 \times 32$): high spatial detail, but semantically naive.
- F_d = Upsampled decoder feature map (same spatial size, e.g., $296 \times 296 \times 32$): semantically aware, spatially blurry.

2. Channel-Average Pooling on BOTH:

- $A_e(h, w) = \frac{1}{C} \sum_c F_e(h, w, c) \rightarrow$ Activity map from encoder ($296 \times 296 \times 1$).
- $A_d(h, w) = \frac{1}{C} \sum_c F_d(h, w, c) \rightarrow$ Activity map from decoder ($296 \times 296 \times 1$).

3. Concatenate Along Channel Axis:

$$A_{\text{cat}} = [A_e; A_d] \in \mathbb{R}^{296 \times 296 \times 2}$$

Now we have a 2-channel map: channel 1 = "what the encoder thinks is important" and channel 2 = "what the decoder thinks is important."

4. **7×7 Convolution (2 input channels $\rightarrow$ 1 output channel):** This convolution learns to COMPARE the encoder's activity with the decoder's activity. It answers: "at locations where BOTH the encoder AND decoder agree something is important $\rightarrow$ this is very likely a vessel."

5. **Sigmoid $\rightarrow$ Attention Mask** (values 0 to 1).

6. Gate the Encoder Features:

$$F_{\text{gated}} = F_e \odot \text{AttentionMask}$$

Only the encoder features that the decoder "approves of" are allowed through.

7. **Concatenate and Feed to Decoder:** The decoder now receives: $[F_d; F_{\text{gated}}]$ – clean, relevant, semantically-filtered spatial details.

Parameters: $7 \times 7 \times 2 \times 1 = 98$ parameters per CSA module. Three CSA modules total = **294 parameters** for ALL attention in the entire skip connection system. This is NEGLIGIBLE compared to the model's 259,960 total parameters.

Analogy: A Security Checkpoint

Imagine an airport security checkpoint.

- The **encoder features** are ALL the passengers trying to board the plane (including some who shouldn't be there — noise, artifacts, lesions).
 - The **decoder features** are like the passenger manifest — a list of who SHOULD be on the plane (vessel locations).
 - **CSA** is the security guard who checks each passenger (encoder feature) against the manifest (decoder guidance). Legitimate passengers (real vessel features) pass through; imposters (noise) are blocked.
-

B5. Innovation 3: The Loss Function — BCE + MCC

B5.1: What Is a Loss Function? (The Basics)

A **loss function** measures "how wrong is the network's prediction compared to the correct answer." It produces a single number:

- **Loss = 0**: Perfect prediction (impossible in practice).
- **Loss = high**: Terrible prediction.

The entire training process is about **minimizing this loss number**. The network adjusts its weights in the direction that reduces the loss (this is gradient descent).

Analogy: Golf Score

The loss is like your golf score — lower is better. Each training step, you slightly adjust your swing (weights) to reduce your score (loss). Over thousands of rounds (epochs), your score improves.

B5.2: Binary Cross-Entropy (BCE) Loss — The Pixel-by-Pixel Checker

For each pixel, the model predicts a probability p_i (a number between 0 and 1):

- $p_i = 0.95 \rightarrow$ "I'm 95% sure this is a vessel."
- $p_i = 0.02 \rightarrow$ "I'm 98% sure this is NOT a vessel."

The ground truth label is y_i :

- $y_i = 1 \rightarrow$ This pixel IS a vessel (as traced by the expert).
- $y_i = 0 \rightarrow$ This pixel is NOT a vessel.

BCE formula:

$$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \right]$$

What does this actually compute?

- If the pixel IS a vessel ($y_i = 1$): loss = $-\log(p_i)$.
 - If $p_i = 0.99$ (model is confident and correct): $-\log(0.99) = 0.01 \rightarrow$ tiny loss 
 - If $p_i = 0.01$ (model is confident and WRONG): $-\log(0.01) = 4.6 \rightarrow$ huge loss 
- If the pixel is NOT a vessel ($y_i = 0$): loss = $-\log(1 - p_i)$.
 - If $p_i = 0.01$ (correctly says not a vessel): $-\log(0.99) = 0.01 \rightarrow$ tiny loss 
 - If $p_i = 0.99$ (wrongly says it IS a vessel): $-\log(0.01) = 4.6 \rightarrow$ huge loss 

In plain English: BCE heavily penalizes confident wrong predictions and rewards confident correct predictions.

The Fatal Flaw of BCE Alone: Class Imbalance

In a retinal image, roughly:

- ~10% of pixels are vessels (foreground).
- ~90% of pixels are background.

What if the network just predicts $p_i = 0.0$ for EVERY pixel (says "nothing is a vessel")?

- It gets the 90% background pixels correct $\rightarrow$ low loss on those.
- It gets the 10% vessel pixels wrong $\rightarrow$ high loss on those.
- But since there are 9 $\times$ more background pixels, the average loss is dominated by the correctly-predicted background. The total loss is still pretty low!

The network learns a lazy shortcut: **"just predict everything as background and your loss will be decent."** This is called the **class imbalance problem** — the network stops trying to find vessels because it's easier to just ignore them.

B5.3: Matthews Correlation Coefficient (MCC) Loss — The Imbalance Killer

What is MCC?

MCC is a metric that evaluates the quality of binary classification using ALL four outcomes:

	Predicted: Vessel	Predicted: Not Vessel
Actual: Vessel	True Positive (TP) ✓	False Negative (FN) ✗
Actual: Not Vessel	False Positive (FP) ✗	True Negative (TN) ✓

- **TP:** Correctly identified vessel pixels.
- **TN:** Correctly identified background pixels.
- **FP:** Background pixels incorrectly called vessels (false alarm).
- **FN:** Vessel pixels incorrectly called background (missed vessels).

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

MCC ranges from **-1** (totally wrong) to **+1** (perfectly correct). A score of **0** means no better than random guessing.

Why is MCC immune to class imbalance?

Remember the lazy network that predicts everything as background?

- $TP = 0$ (no vessels detected).
- $FN = \text{\$ all vessel pixels}$ (all vessels missed).
- $FP = 0$ (never false-alarmed since it never predicts vessel).
- $TN = \text{\$ all background pixels}$ (correctly predicted).

Plugging in: $MCC = \frac{0 \times TN - 0 \times FN}{\sqrt{\dots}} = 0$

MCC gives this lazy network a score of exactly 0 (random chance). It cannot be cheated! Compare this to BCE, which would give a deceptively low loss.

Making MCC Differentiable (The Soft Approximation)

Standard MCC uses hard binary predictions (pixel is either 0 or 1). But neural networks need **gradients** for learning, and hard 0/1 decisions have zero gradient (you can't take the derivative of a step function).

SA-UNetv2's trick: Replace the hard binary values with the soft probability outputs $p_i \in [0, 1]$

$$TP_{\text{soft}} = \sum_i p_i \cdot y_i$$

Intuition: If pixel i is a true vessel ($y_i = 1$) and the model predicts $p_i = 0.8$, this contributes 0.8 to TP instead of a hard 1. The "softness" allows gradients to flow.

Similarly:

$$FP_{\text{soft}} = \sum_i p_i \cdot (1 - y_i)$$

$$FN_{\text{soft}} = \sum_i (1 - p_i) \cdot y_i$$

$$TN_{\text{soft}} = \sum_i (1 - p_i) \cdot (1 - y_i)$$

The MCC loss is then:

$$\mathcal{L}_{MCC} = 1 - \text{MCC}_{\text{soft}}$$

Minimizing $\mathcal{L}_{MCC}$ = Maximizing MCC = Better balanced classification.

B5.4: The Combined Loss — Why 50/50?

$$\mathcal{L}_{\text{total}} = 0.5 \times \mathcal{L}_{BCE} + 0.5 \times \mathcal{L}_{MCC}$$

- **BCE** gives strong pixel-level gradient signals (makes the model locally accurate).
- **MCC** ensures global balance (prevents the model from ignoring the minority vessel class).
- The paper tested multiple ratios (0.7:0.3, 0.3:0.7, 1.0:0.0, 0.0:1.0) and found **50/50 produces the best F1 score**.

Why not use Dice Loss instead of MCC?

Your professor might ask this. **Dice Loss** is another popular imbalance-handling loss, but it only considers TP , FP , and FN — it **completely ignores** TN (True Negatives). In fundus images, 90% of pixels are background, and correctly classifying background is actually clinically important (you don't want false vessel detections near lesions). MCC considers all four quadrants, making it strictly more informative.

PART C: TRAINING MECHANICS — HOW THE MODEL ACTUALLY LEARNS

C1. The Training Loop (What Happens in Each Epoch)

An **epoch** = one complete pass through all training images.

For each epoch:

1. **Load a batch** of images (e.g., 8 DRIVE images or 2 STARE images).
 2. **Forward pass:** Feed images through the network → get predicted probability maps.
 3. **Compute loss:** Compare predictions against ground truth using $0.5 \cdot BCE + 0.5 \cdot MCC$.
 4. **Backpropagation:** Calculate the gradient of the loss with respect to every weight in the network (this tells you: "if I increase this weight by a tiny amount, does the loss go up or down?").
 5. **Update weights:** Nudge each weight in the direction that reduces the loss (using the Adam optimizer).
 6. **Repeat** for all batches in the dataset.
 7. **Validation:** After processing all training batches, evaluate on a held-out validation set (10% of training data). The model does NOT learn from validation data — it's purely for monitoring.
-

C2. The Optimizer: Adam

Adam (Adaptive Moment Estimation) is the algorithm that decides HOW MUCH to nudge each weight.

Simple gradient descent uses a fixed learning rate for all weights. Adam is smarter:

- It maintains a **per-weight moving average** of past gradients (momentum).
- Weights that have been receiving large, consistent gradients get smaller updates (they're already moving in the right direction).
- Weights that have been receiving small, noisy gradients get larger updates (they need more help).

Initial learning rate: 0.001 — this is the base step size. Adam adapts it per-weight from here.

C3. Learning Rate Scheduler: ReduceLROnPlateau

As training progresses, the model gets closer to optimal. Large learning rates that were helpful early on now cause the model to "overshoot" the optimum (like trying to park a car at full speed).

ReduceLROnPlateau monitors the **validation loss**:

- If validation loss hasn't improved for **10 consecutive epochs** (patience = 10), it **halves** the learning rate ($\times 0.5$).
- Minimum learning rate: 10^{-6} (never goes below this).

Analogy: Searching for a Valley

Imagine walking downhill in fog to find the lowest point in a valley. At first, you take big steps (large learning rate) to quickly get into the valley. As you sense you're near the bottom (loss plateaus), you take smaller and smaller steps to precisely find the absolute lowest point.

C4. Early Stopping

If validation loss doesn't improve for **20 consecutive epochs**, training stops automatically. This prevents **overfitting** — the point where the model memorizes the training images instead of learning generalizable patterns.

Our results:

- DRIVE: Early stopped at **Epoch 138** (out of max 150). Best validation loss: **0.1788**.
- STARE: Completed all **100 epochs**. Best validation loss: **0.1446**.

C5. Data Augmentation

We only have 20 training images (DRIVE) or 16 training images (STARE). That's tiny! Deep learning usually needs thousands of images.

Data augmentation artificially creates variations of existing images:

- **Horizontal flip**: Mirror the image left-to-right.
- **Vertical flip**: Mirror the image top-to-bottom.

- **90° rotations:** Rotate by 90, 180, 270 degrees.
- **Color jitter:** Slightly randomize brightness, contrast.

Each epoch, random augmentations are applied, so the network sees slightly different versions of the same images each time. This prevents memorization and improves generalization.

PART D: EVALUATION METRICS — WHAT EACH NUMBER MEANS

D1. The Confusion Matrix (Foundation of Everything)

After the model predicts vessel maps for all test images, we threshold at $\tau = 0.5$ (any pixel with probability > 0.5 is called "vessel", otherwise "background") and count:

	Predicted: Vessel	Predicted: Background
Actual: Vessel	TP (correctly found vessels)	FN (missed vessels)
Actual: Background	FP (false alarms)	TN (correctly ignored background)

Every evaluation metric is just a different way of combining these four numbers.

D2. Each Metric Explained

Sensitivity (also called Recall)

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

"Of all the real vessels in the image, what percentage did we actually find?"

- High sensitivity = we're catching most vessels (good for diagnosis — don't miss a diseased vessel!).
- Our STARE model: **83.38%** → we detect 83% of all vessel pixels.

Specificity

$$\text{Specificity} = \frac{TN}{TN + FP}$$

"Of all the real background pixels, what percentage did we correctly leave alone?"

- High specificity = very few false alarms.
- Our STARE model: **98.50%** → only 1.5% of background was falsely called "vessel."

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

"What percentage of ALL pixels (vessel + background) did we get right?"

- Our STARE model: **97.40%** → 97.4% of all pixels are correctly classified.
- ⚠ Accuracy is misleading with class imbalance! A model that predicts 100% background gets ~90% accuracy automatically.

F1-Score (Dice Coefficient)

$$F1 = \frac{2 \times TP}{2 \times TP + FP + FN}$$

"The harmonic mean of Precision and Recall – a balanced score that punishes both false alarms AND missed vessels."

- Our STARE model: **82.44%** → a strong, balanced score.
- F1 is widely considered the single most important metric for segmentation.

Jaccard Index (IoU – Intersection over Union)

$$\text{IoU} = \frac{TP}{TP + FP + FN}$$

"Of the total pixels that are either truly vessels or predicted as vessels, how many are in both categories?"

- Mathematically related to F1: $\text{IoU} = \frac{F1}{2 - F1}$

- Our STARE model: **70.22%**

Matthews Correlation Coefficient (MCC)

$$\text{MCC} = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

"The gold-standard balanced metric that considers ALL four quadrants and cannot be fooled by class imbalance."

- Our STARE model: **81.08%**

AUC-ROC (Area Under the Receiver Operating Characteristic Curve)

Instead of using a single threshold ($\tau = 0.5$), AUC sweeps through ALL possible thresholds from 0 to 1, plotting Sensitivity vs. (1 - Specificity) at each one.

- **AUC = 1.0**: Perfect model — at some threshold, it achieves 100% sensitivity AND 100% specificity.
- **AUC = 0.5**: Random guessing.
- Our STARE model: **98.69%** → the model's probability outputs separate vessels from background almost perfectly.

PART E: OUR REPRODUCTION RESULTS — THE NUMBERS

E1. DRIVE Dataset Results

Metric	Published Paper (ISBI 2026)	Our Reproduction	Difference
F1-Score	82.82%	80.09%	-2.73%
Specificity	98.28%	98.12%	-0.16%
Accuracy	96.98%	96.53%	-0.45%
AUC-ROC	98.71%	98.00%	-0.71%

Metric	Published Paper (ISBI 2026)	Our Reproduction	Difference
Sensitivity	83.64%	80.20%	-3.44%
MCC	81.27%	78.32%	-2.95%

Why is there a 2.7% gap?

The authors used an **offline pre-augmented dataset** on Kaggle with hundreds of pre-computed elastic deformation grids saved to disk. We used simple online augmentation (random flips and rotations). The elastic deformations create more topological variety in thin vessel shapes, accounting for the gap.

E2. STARE Dataset Results

Metric	Published Paper (ISBI 2026)	Our Reproduction	Difference
F1-Score	82.81%	82.44%	-0.37%
Jaccard	70.82%	70.22%	-0.60%
Specificity	98.71%	98.50%	-0.21%
Accuracy	97.83%	97.40%	-0.43%
MCC	81.79%	81.08%	-0.71%
AUC-ROC	99.13%	98.69%	-0.44%

Our STARE reproduction is within **0.37%** of the paper — essentially matching it!

PART F: VIVA Q&A — COMPREHENSIVE ANSWERS

Q: "What is the overall architecture? Describe the flow from input to output."

"The input is a retinal fundus image (RGB, 3 channels), zero-padded to dimensions divisible by 8. It passes through a 4-level U-Net encoder-decoder:

*The **encoder** has 4 stages with channel counts [16, 32, 48, 64]. Each stage applies two Conv Blocks (Conv3×3 → DropBlock → GroupNorm → SiLU) followed by 2×2 Max Pooling, progressively halving the spatial dimensions while increasing semantic understanding.*

*At the **bottleneck** (64 channels, smallest spatial size), a Spatial Attention module identifies the most important spatial locations.*

*The **decoder** mirrors the encoder, using Transposed Convolutions for upsampling. At each level, a Cross-Scale Spatial Attention (CSA) module filters the skip connection features using decoder guidance before concatenation.*

Finally, a 1×1 convolution with sigmoid produces a single-channel probability map, which is cropped back to the original dimensions for evaluation."

Q: "What datasets did you use?"

"Two standard retinal vessel segmentation benchmarks:

- 1. **DRIVE** (Digital Retinal Images for Vessel Extraction): 40 images, 20 train / 20 test, 584×565 padded to 592×592.*
 - 2. **STARE** (STructured Analysis of the Retina): 20 images (Hoover benchmark), 16 train / 4 test, 700×605 padded to 704×704."*
-

Q: "What makes this model lightweight?"

"Three design choices: (1) Aggressive channel reduction to [16, 32, 48, 64] instead of the standard [64, 128, 256, 512], cutting parameters by 99% compared to standard U-Net. (2) The CSA attention modules add only 98 parameters each — nearly free. (3) The entire model is just 259,960 parameters and 21.19 GFLOPs — 120× smaller than standard U-Net with comparable accuracy."

Q: "Why is your DRIVE result 2.7% lower than the paper?"

"The authors used offline elastic deformation augmentation — they pre-generated hundreds of warped image variants and saved them to disk before training. This creates much greater topological diversity in vessel shapes than our online random

augmentations (flips and rotations). We verified this by noting that our STARE results (where augmentation differences are smaller) match within just 0.37%."

Q: "What is the clinical significance of this work?"

"Retinal fundus imaging is the cheapest, most accessible diagnostic modality for screening diabetic retinopathy, glaucoma, and cardiovascular risk. Accurate automated vessel segmentation enables:

1. Mass screening in resource-limited settings (rural clinics, developing nations) where ophthalmologists are scarce.
 2. Quantitative vessel biomarker extraction (arteriolar narrowing, tortuosity index, fractal dimension) for disease staging.
 3. Integration into portable screening devices — SA-UNetv2's 0.26M parameters can run on smartphones and edge devices in real-time (~20ms/image on GPU, ~1s on CPU)."
-

Q: "What will you do next?"

"Phase 2 implements novel extensions from the ACE-ProtoNet companion paper (Medical Image Analysis 2026):

1. **Centerline Dice (cIDice):** A topological loss that penalizes vascular tree disconnections by comparing vessel skeletons.
2. **Uncertainty Calibration:** Using Shannon entropy maps to visualize and quantify prediction confidence on ambiguous vessel boundaries.
3. **Uncertainty-Weighted Loss:** Dynamically increasing the loss weight on uncertain, hard boundary pixels to push accuracy higher."